

```
name: Greetings

on: [pull_request_target, issues]

jobs:
  greeting:
    runs-on: ubuntu-latest
    permissions:
      issues: write
      pull-requests: write
    steps:
      - uses: actions/first-interaction@v1
        with:
          repo-token: ${ secrets.GITHUB_TOKEN }
          issue-message: "Message that will be displayed on
users' first issue"
          pr-message: "Message that will be displayed on users'
first pull request"

Show less ^
```

This GitHub Actions workflow automatically greets contributors the **first time** they open an **Issue** or **Pull Request** in your repository.

Here's a breakdown:

</> YAML



name: Greetings

ChatGPT Upgrade

Share ...

- The workflow name displayed in the **Actions** tab.

</> YAML



on: [pull_request_target, issues]

The workflow is triggered when:

- `issues` → someone opens or edits an issue.

- `pull_request_target` → someone creates or updates a pull request against the base repository.

Why `pull_request_target` instead of `pull_request` ?

`pull_request_target` runs in the context of the target repository, allowing access to `GITHUB_TOKEN` with write permissions. This is useful because the workflow needs permission to comment on the PR.

⌕ YAML



```
jobs:
  greeting:
```

Defines a job named `greeting`.

⌕ YAML



```
runs-on: ubuntu-latest
```

The job runs on GitHub's latest Ubuntu runner.

⌕ YAML



```
permissions:
  issues: write
  pull-requests: write
```

Grants the workflow permission to write comments.

Permission	Purpose
<code>issues: write</code>	Comment on issues
<code>pull-requests: write</code>	Comment on pull requests

</> YAML



```
steps:
```

The sequence of actions to execute.

</> YAML



```
- uses: actions/first-interaction@v1
```

Uses GitHub's official **First Interaction** action.

This action checks whether the contributor has previously:

- Opened an issue
- Opened a pull request

If it's their **first** interaction, it posts a welcome message.

</> YAML



```
with:
```

Inputs for the action.

Authentication

</> YAML



```
repo-token: ${ secrets.GITHUB_TOKEN }
```

Uses GitHub's automatically generated token.

This token lets the workflow:

- Add comments
- Access repository information

No need to create your own Personal Access Token.

First Issue Message

</> YAML



```
issue-message: "Message that will be displayed on users' first issue"
```

Example:

</> YAML



```
issue-message: |
  🙌 Thanks for opening your first issue!

  We appreciate your feedback.
  A maintainer will review it soon.
```

First Pull Request Message

</> YAML



```
pr-message: "Message that will be displayed on users' first pull request"
```

Example:

</> YAML



```
pr-message: |
  🎉 Thanks for your first contribution!

  Please ensure:
  - ✅ Tests pass
  - ✅ Code follows our style guide
  - ✅ Documentation is updated if needed

  Happy contributing!
```

Example Workflow

</> YAML



```
name: Greetings
```

```

on:
  issues:
    types: [opened]
  pull_request_target:
    types: [opened]

jobs:
  greeting:
    runs-on: ubuntu-latest

    permissions:
      issues: write
      pull-requests: write

    steps:
      - uses: actions/first-interaction@v1
        with:
          repo-token: ${{ secrets.GITHUB_TOKEN }}

          issue-message: |
            🙌 Welcome!

            Thanks for opening your first issue.
            We appreciate your contribution and will review it soon.

          pr-message: |
            🎉 Thank you for your first Pull Request!

            Please make sure:
            - ✅ CI passes
            - ✅ Documentation is updated if needed
            - ✅ Code follows project conventions

            Happy coding! 🚀

```

How It Works

1. A user opens their **first issue**.
2. GitHub triggers the workflow.
3. The `actions/first-interaction` action checks if the user has interacted with the repository before.
4. If it's their first issue, it posts the `issue-message`.
5. If it's their first pull request, it posts the `pr-message`.
6. On subsequent issues or pull requests by the same user, **no greeting is posted**.

When to Use This Workflow

This workflow is useful for open-source repositories because it:

- Welcomes new contributors.
- Shares contribution guidelines automatically.
- Improves the first-time contributor experience.
- Reduces the need for maintainers to manually greet contributors.



difference between these two

on:

issues:

types: [opened]

pull_request_target:

types: [opened]

on: [pull_request_target, issues]

The main difference is **which events trigger the workflow**. The second syntax is shorthand, while the first is more specific.

Syntax	Trigger	Recommended?
<code>on: [pull_request_target, issues]</code>	All activity types for both events	✗ Usually no
<code>issues: types: [opened]</code>	Only when an issue is opened	✓ Yes
<code>pull_request_target: types: [opened]</code>	Only when a PR is opened	✓ Yes

1. Shorthand syntax

</> YAML



```
on: [pull_request_target, issues]
```

This means:

 **YAML**



```
on:
  pull_request_target:
  issues:
```

Since no `types` are specified, GitHub triggers the workflow for **all default activity types** of those events.

For example, the `issues` event can be triggered by:

- opened
- edited
- deleted
- closed
- reopened
- assigned
- unassigned
- labeled
- unlabeled
- milestoned
- demilestoned
- pinned
- unpinned
- locked
- unlocked
- transferred
- etc.

Similarly, `pull_request_target` can trigger on activities such as:

- opened
- synchronize (new commits pushed)
- reopened
- edited
- labeled
- unlabeled
- ready_for_review
- converted_to_draft
- review_requested
- review_request_removed
- closed

- and others.

So your workflow may run **many more times than you expect**.

2. Specifying activity types

</> YAML 

```
on:
  issues:
    types: [opened]
  pull_request_target:
    types: [opened]
```

Now the workflow runs **only once** when:

- a new issue is opened
- a new pull request is opened

It **will not** run when:

- ✗ Someone edits the issue
 - ✗ Someone labels the issue
 - ✗ Someone reopens the issue
 - ✗ Someone pushes more commits to the PR (`synchronize`)
 - ✗ Someone changes the PR title
-

Example

Suppose someone creates a PR.

Using

</> YAML 

```
on: [pull_request_target]
```

Workflow runs.

Then they push another commit.

Workflow runs **again**.

Then they edit the PR description.

Workflow runs **again**.

Then they add a label.

Workflow runs **again**.

Using

</> YAML



```
on:
  pull_request_target:
    types: [opened]
```

Workflow runs only when the PR is initially opened.

Pushing more commits? ❌ No run.

Editing description? ❌ No run.

Adding labels? ❌ No run.

Which should you use?

For a greeting workflow like `actions/first-interaction`, use:

</> YAML



```
on:
  issues:
    types: [opened]
  pull_request_target:
    types: [opened]
```

This is more efficient because:

- The workflow only runs when it needs to.
- It avoids unnecessary workflow executions.
- It reduces GitHub Actions usage and clutter in the Actions tab.

In general, use the `types` filter whenever you only care about specific actions, and use the shorthand only if you intentionally want the workflow to react to all activity types for those events.

